

Audit & Feuille de route stratégique — QuizExam BF

*Préparé le 7 juillet 2026 — par Claude, à la demande de
Giovanni BAMOGO*

0. Méthodologie — mise à jour après lecture du code réel

Après ton premier message, je n'avais accès qu'à la coquille HTML du site (rendu 100% client-side). Tu m'as ensuite donné

`git@github.com:Giova03/quizexam-bf1.git` — je l'ai cloné et j'ai lu le code réel (Prisma, routes API, config Next.js, worklog de développement de 1116 lignes, historique des correctifs). Tout ce qui suit remplace mes suppositions initiales par des faits vérifiés dans ton dépôt.

Bonne nouvelle d'abord : le projet est bien plus avancé que ce qu'un simple coup d'œil au site ne laissait supposer. Stack solide (Next.js 16, React 19, Prisma/PostgreSQL, next-auth, Zustand, Tailwind v4), et une grande partie de ma liste de 124

fonctionnalités plus bas est **déjà construite** : forum, groupes d'étude, événements, blog, parrainage, certificats, IA tuteur, export Anki (répétition espacée SM-2), mode compétition, défi quotidien, classement, API publique documentée avec rate limiting, PWA avec service worker et synchronisation hors-ligne (IndexedDB), panneau d'accessibilité (contraste élevé, police dyslexie, lecteur d'écran), notifications push, visite guidée d'onboarding, modération, analytics admin. Je corrige donc mes recommandations précédentes qui supposaient l'absence de ces éléments.

1. Ce qui ne va pas — constat réel basé sur le code

● Critique — à traiter immédiatement

Le dépôt GitHub est public et contient des fichiers qui ne devraient jamais y être.

- `upload/` (42 fichiers trackés, ~186 Mo) contient des PDF de cours, une conversation WhatsApp exportée, un fichier `Giovanni.txt` personnel, et surtout un livre scanné manifestement piraté (nom de fichier se terminant par "Z-Library"). Avoir un livre sous copyright dans un dépôt **public** est

un vrai risque légal, pas juste une question de propreté.

- `skills/` (1004 fichiers trackés, ~61 Mo) est une bibliothèque d'agents IA totalement étrangère à ton projet quiz (dream-interpreter, stock-analysis, gaokao-*, etc.) — probablement copiée par erreur par l'agent Z.ai Code, à supprimer.
- `db/custom.db` (base SQLite locale) est trackée alors que ton `.gitignore` dit `db/*.db` — la règle a été ajoutée après coup, le fichier existant n'a jamais été retiré.
- Conséquence : dépôt de 408 Mo, historique `.git` de 154 Mo, clone/déploiement plus lents que nécessaire.

Action recommandée : rendre le dépôt privé dès maintenant en attendant le nettoyage, puis :

```
git rm -r --cached upload/ skills/ db/
echo -e "upload/\nskills/\ndb/\n" >>
.gitignore
git commit -m "Retrait des fichiers non
liés au code (contenu source, bibliothèque
d'agents, DB locale)"
```

Comme ces fichiers restent dans l'historique même après suppression, utilise ensuite `git filter-repo` (ou BFG Repo-Cleaner) pour purger le blob du livre piraté de tout l'historique avant de remettre le dépôt

en public — sinon il reste récupérable via les anciens commits.

`next.config.ts` désactive les garde-fous de qualité :

```
typescript: { ignoreBuildErrors: true },
eslint: { ignoreDuringBuilds: true },
reactStrictMode: false,
```

Concrètement, ton build Next.js **ne peut jamais échouer** à cause d'une erreur TypeScript ou ESLint — ce qui explique exactement le pic que j'ai vu dans ton `worklog.md` : 112 erreurs TypeScript et 10 erreurs de lint accumulées avant d'être corrigées d'un coup. Sans ce filet, une erreur silencieuse peut partir en production sans que personne ne le sache. Je recommande de retirer ces deux lignes (ou au moins de les faire passer à `false`) et de réactiver `reactStrictMode`, qui détecte des bugs React avant qu'ils n'arrivent en prod.

Aucune CI/CD. Pas de dossier `.github/workflows` : aucun test, lint ou build n'est vérifié automatiquement avant qu'un changement parte en production sur Vercel. Des fichiers de test existent bien (`sm2.test.ts`, `cache.test.ts`, `favorites.test.ts` ...) mais il n'y a même pas de script `test` dans `package.json` — ils ne sont donc jamais exécutés dans le flux normal.

● Modéré

Contrôle d'autorisation manquant sur PATCH

`/api/sessions/[id]/answers/[answerId]` : la route vérifie que la réponse appartient bien à la session, mais ne vérifie pas que l'utilisateur authentifié est le propriétaire de cette session. Quelqu'un qui connaît (ou intercepte) les identifiants pourrait modifier les réponses d'un examen qui n'est pas le sien.

Contenu très déséquilibré par niveau — j'ai vérifié directement dans les 66 banques générées : 1 seule banque COLLEGE (≈ BEPC), 7 LYCEE, 1 TERM (≈ Bac Terminale), contre 31 LICENCE et 25 TOUS. Ça confirme précisément ta demande : la plateforme cible aujourd'hui presque exclusivement les concours niveau licence/professionnels, avec quasiment rien pour le BEPC et le Bac. Bonne nouvelle : le champ level existe déjà dans le schéma Prisma — la fondation technique est là, il manque surtout le contenu et une navigation dédiée par niveau dans l'interface.

Inscription aux événements stockée uniquement côté client (localStorage) selon le schéma — perdue si l'utilisateur change d'appareil ou vide son cache. À migrer vers une vraie table de participation en base.

Dépendance à un fournisseur IA unique (z-ai-web-dev-sdk) pour le tuteur IA et la génération de questions — à surveiller : coût à l'échelle, fiabilité, et

risque si ce service change ses conditions ou tombe en panne.

● Positif — déjà bien fait, à ne pas casser

- **Le score n'est jamais calculé côté client :**

`isCorrect` est recalculé côté serveur en comparant à `correctAnswer` stocké en base (`sessions/[id]/answers/[answerId]/route.ts`).
Pas de faille de triche évidente ici.

- **PWA solide** : service worker avec stratégies de cache différenciées (statique vs API), file de synchronisation en arrière-plan, gestionnaire offline complet basé sur IndexedDB avec fallback mémoire.
 - `.env*` **correctement ignoré** — aucun secret trouvé dans le dépôt cloné.
-

2. Architecture technique — pour que le projet dure et encaisse la charge

Rendu & performance

- Next.js en mode hybride : SSG/ISR pour les pages publiques et les bancs de questions (contenu statique régénéré périodiquement), client-side uniquement pour l'examen interactif lui-même.

- Compression et lazy-loading des images, `next/image` , fonts en local (pas de requête externe bloquante).
- Mode "données réduites" : détecter une connexion lente (`navigator.connection`) et servir une version allégée.
- PWA (service worker + manifest) pour un fonctionnement hors-ligne partiel et une installation sur écran d'accueil.

Données & fiabilité (*mis à jour : tu es déjà sur Postgres/Prisma, bon choix pour ta hiérarchie Niveau > Filère > Concours > Matière > Question*)

- Ajoute Redis (ou l'équivalent Vercel KV) pour le rate limiting persistant — actuellement `rate-limit.ts` est un limiteur en mémoire, donc il repart à zéro à chaque redéploiement/redémarrage de fonction serverless et n'est pas partagé entre plusieurs instances.
- Le score côté serveur est déjà bien géré (voir section 1) — assure-toi juste que toute future route qui touche au score suit le même principe.
- Sauvegardes automatiques régulières de la base Postgres + un vrai test de restauration (un backup jamais testé n'est pas un backup) — à vérifier côté hébergeur de ta base.

Qualité & observabilité (*le vrai chantier prioritaire, voir section 1*)

- Retirer `ignoreBuildErrors / ignoreDuringBuilds` de `next.config.ts` et réactiver `reactStrictMode`.
- Ajouter un script `"test"` dans `package.json` (les fichiers de test existent déjà, ils ne sont juste jamais lancés) et une CI GitHub Actions minimale : `tsc --noEmit, eslint ., bun test, next build` — bloquant avant merge/déploiement.
- Sentry (ou équivalent, gratuit en petit volume) pour choper les erreurs en prod en temps réel — rien n'est en place aujourd'hui.
- Environnement de staging séparé de la prod avant chaque mise en ligne (Vercel le permet nativement via les preview deployments par branche — à activer/formaliser si pas déjà utilisé systématiquement).

Scalabilité (**le pic de trafic la veille d'un concours, c'est garanti**)

- Cache agressif sur les bancs de questions (contenu qui change peu).
- CDN (déjà natif avec Vercel, bon point).
- Limiter les requêtes lourdes non essentielles pendant les pics (ex: analytics en asynchrone, pas bloquant).

3. Segmentation par niveau – le cœur de ta demande

Bonne nouvelle confirmée dans le code :

`QuestionBank.level` existe déjà dans ton schéma Prisma avec les valeurs `COLLEGE`, `LYCEE`, `TERM`, `LICENCE`, `TOUS`. Il ne s'agit donc pas de créer cette fonctionnalité de zéro, mais de :

1. Renommer/aligner ces valeurs sur la terminologie réelle burkinabè (`COLLEGE` → "BEPC", `LYCEE / TERM` → "Bac" avec sous-catégorie série, `LICENCE` → concours administratifs/professionnels).
2. Construire la navigation UI dédiée (aujourd'hui apparemment absente – aucune mention de "BEPC"/"Bac"/"Licence" trouvée dans les composants frontend).
3. Combler le contenu : 1 banque niveau collège et 1 niveau terminale contre 31 niveau licence, il faut du contenu BEPC/Bac en priorité.

Structure proposée :

```
Niveau d'étude
├─ BEPC (collège)
├─ Baccalauréat (par série : A, C, D, E,
G, F...)
└─ Licence / Concours administratifs et
```

```
professionnels
  └─ Fonction publique (ENAM, ENEP,
    Douane, Police, Gendarmerie, Santé/ENSP,
    Magistrature...)
    └─ Grandes écoles (IPD/AOS, écoles
      d'ingénieurs, etc.)
      └─ (extensible plus tard : Master,
        concours internationaux, professionnels)
        └─ Matière
          └─ Chapitre / thème
            └─ Banc de questions
```

Fonctionnalités clés associées : test de positionnement initial pour orienter automatiquement l'utilisateur, filtre multi-critères (niveau + série + matière + année + difficulté), et une progression débloquée par palier (comme des chapitres qui s'ouvrent au fur et à mesure).

4. Plus de 100 fonctionnalités pour rendre la plateforme incontournable

 **Important après lecture du code** : cette liste reste volontairement complète (elle sert de référence long terme), mais un nombre significatif d'items sont **déjà implémentés** dans ton dépôt. Confirmé dans le code : forum par matière (#50), groupes d'étude avec code d'invitation (#49), calendrier d'événements (#12), blog/articles (#122-ish), système de parrainage

(#53/#76), certificats téléchargeables (#23/#47), IA tuteur conversationnel (#26), export Anki / répétition espacée SM-2 (#121), mode compétition, défi quotidien, classement (#42), API publique documentée avec rate limiting, PWA + mode hors-ligne complet (#78), panneau d'accessibilité – contraste élevé, police dyslexie, lecteur d'écran, taille de police (#81-84), notifications push (#103), visite guidée d'onboarding (#112), panneau de modération (#57), analytics admin avec heatmap (#59-67). Les items ci-dessous sans confirmation dans le code sont probablement les vrais manques à prioriser – notamment tout ce qui touche à la segmentation par niveau (section 3), à la gamification XP/badges/ligues (catégorie D), et à la génération IA de QCM à partir de PDF côté utilisateur final (une route `generate-qcm / upload-pdf` existe déjà côté admin – à vérifier si elle est exposée aux enseignants/contributeurs ou réservée à toi).

A. Structuration pédagogique (12)

1. Arborescence Niveau > Filière/Série > Concours > Matière > Chapitre
2. BEPC : toutes les matières du collège
3. Bac : séparation par série avec matières et coefficients réels
4. Concours administratifs/professionnels burkinabè (ENAM, ENEP, IFTIC, Police, Douane,

Santé, Magistrature...)

5. Concours des grandes écoles
6. Filtre multi-critères (niveau, série, matière, année, difficulté)
7. Test de positionnement initial
8. Progression débloquée par palier de maîtrise
9. Bibliothèque d'annales officielles classées par année/session
10. Fiches de cours résumées par matière/niveau
11. Glossaire des termes techniques par matière
12. Calendrier officiel des concours BF avec alertes d'inscription

B. Examen blanc réaliste (12)

13. Chronomètre géré côté serveur (anti-triche)
14. Barème fidèle au concours réel (coefficients, notation négative si applicable)
15. Mode "conditions réelles" (plein écran, blocage copier-coller)
16. Détection de changement d'onglet avec avertissement
17. Sauvegarde automatique en cas de coupure réseau/électricité
18. Reprise de session après déconnexion
19. Questions/réponses mélangées aléatoirement à chaque tentative

- 20. Correction détaillée immédiate avec explication par question
- 21. Historique complet des examens passés avec comparatif de progression
- 22. Séparation claire mode "entraînement" vs mode "examen noté"
- 23. Export du bulletin de résultats en PDF
- 24. Estimation du rang probable au concours réel (basé sur les statistiques historiques)

C. Intelligence artificielle réellement fonctionnelle (14)

- 25. Génération automatique de QCM à partir d'un cours/PDF uploadé
- 26. Tuteur IA conversationnel disponible en continu pour expliquer une notion
- 27. Correction automatique de rédactions/dissertations avec feedback argumenté
- 28. Détection des lacunes et génération d'un plan de révision personnalisé
- 29. Recommandation adaptative des prochains quiz selon le niveau réel
- 30. Explication pédagogique générée pour chaque question ratée
- 31. Prédiction du score probable au concours réel

- 32. Résumé vocal des cours (écoute en déplacement)
- 33. Chatbot d'aide à l'inscription et à l'orientation vers le bon concours
- 34. Analyse des fautes récurrentes en français/orthographe avec exercices ciblés
- 35. Génération automatique de fiches de révision à partir des erreurs fréquentes
- 36. Simulateur d'oral IA avec feedback sur l'élocution (concours à épreuve orale)
- 37. Détection anti-triche par analyse comportementale (temps de réponse suspect)
- 38. Assistant IA pour les contributeurs/enseignants pour créer des bancs de questions plus vite

D. Gamification & motivation (10)

- 39. Points d'expérience (XP) et niveaux
- 40. Badges de réussite par matière/objectif
- 41. Séries de jours consécutifs (streaks) avec rappels
- 42. Classements par établissement, ville ou national
- 43. Ligues hebdomadaires avec montée/descente
- 44. Défis quotidiens/hebdomadaires
- 45. Récompenses virtuelles débloquables (avatars, thèmes)
- 46. Objectifs personnalisés avec barres de progression

- 47. Certificats de réussite
téléchargeables/partageables
- 48. Mode "duel" en temps réel entre deux utilisateurs

E. Social & communauté (10)

- 49. Groupes d'étude par concours/ville/établissement
- 50. Forum de questions/réponses entre candidats
- 51. Partage des résultats sur les réseaux sociaux
- 52. Défis entre amis
- 53. Parrainage (bonus pour les deux)
- 54. Commentaires modérés sous chaque quiz
- 55. Mentorat entre anciens admis et nouveaux candidats
- 56. Fil d'actualité des réussites de la communauté
- 57. Signalement communautaire des questions erronées
- 58. Sondages sur les sujets à approfondir

F. Suivi de progression & analytics (10)

- 59. Tableau de bord personnel détaillé par matière
- 60. Graphique d'évolution du niveau dans le temps
- 61. Comparaison anonymisée avec la moyenne des autres candidats
- 62. Rapport hebdomadaire par email/notification
- 63. Alerte sur les matières faibles

- 64. Suivi du temps de révision total et objectifs hebdomadaires
- 65. Estimation du temps nécessaire avant d'être "prêt"
- 66. Historique exportable (PDF/Excel)
- 67. Heatmap des heures de révision les plus productives
- 68. Indicateur de régularité/assiduité

G. Modèle économique (8)

- 69. Offre freemium avec bancs limités gratuits
- 70. Abonnement premium mensuel/annuel
- 71. Paiement Mobile Money natif (Orange Money, Moov Money)
- 72. Offres groupées pour établissements scolaires
- 73. Système de dons/soutien au projet
- 74. Marketplace de contenu (enseignants vendent leurs bancs de questions)
- 75. Publicité non-intrusive en version gratuite (à valider)
- 76. Réduction d'abonnement via parrainage

H. Accessibilité & inclusivité (8)

- 77. Mode faible consommation de données
- 78. Fonctionnement hors-ligne (PWA) avec synchronisation différée

- 79. Multilingue (français, puis langues nationales : mooré, dioula, fulfuldé)
- 80. Poids de page réduit pour téléphones d'entrée de gamme
- 81. Mode contraste élevé (malvoyants)
- 82. Compatibilité lecteur d'écran
- 83. Version quiz par SMS/USSD pour zones sans internet
- 84. Tailles de police ajustables

I. Administration & contenu (8)

- 85. Back-office complet de gestion des questions
- 86. Import en masse via Excel/CSV
- 87. Rôles (admin, contributeur/enseignant, modérateur, utilisateur)
- 88. Workflow de validation avant publication
- 89. Statistiques par question (taux de réussite, temps moyen) pour ajuster la difficulté
- 90. Historique des modifications des bancs de questions
- 91. Duplication/traduction rapide de contenu
- 92. Journal d'audit des actions admin

J. Sécurité, fiabilité & anti-fraude (10)

- 93. Validation systématique des réponses côté serveur

- 94. Authentification sécurisée (JWT + refresh token, 2FA optionnel)
- 95. Chiffrement des données sensibles
- 96. Rate limiting sur les API
- 97. Sauvegardes automatiques régulières et testées
- 98. Plan de reprise après incident
- 99. Monitoring des erreurs en temps réel
- 100. Tests automatisés avant chaque déploiement
- 101. Environnement de staging séparé
- 102. Conformité protection des données, particulièrement pour les mineurs (candidats BEPC)

K. Notifications & engagement (6)

- 103. Rappel quotidien de révision
- 104. Alerte avant date limite d'inscription à un concours
- 105. Rappel de reprise après inactivité
- 106. Notification de nouveau contenu pour le concours suivi
- 107. Résumé hebdomadaire par email
- 108. Notification de changement de classement

L. Design & UX moderne (10)

- 109. Refonte visuelle : palette contemporaine, typographie lisible, dark mode natif

- 110. Micro-interactions et animations sur feedback immédiat
- 111. Interface mobile-first repensée
- 112. Onboarding interactif pour nouveaux utilisateurs
- 113. Dashboard personnalisable (widgets réorganisables)
- 114. Iconographie propre au contexte burkinabè
- 115. Mode focus pendant l'examen (interface épurée, sans distraction)
- 116. Design system cohérent (composants réutilisables)
- 117. Accessibilité WCAG AA a minima
- 118. Chargement perçu optimisé (skeleton loaders)

M. Contenu enrichi (6)

- 119. Vidéos explicatives courtes par concept clé
- 120. Podcasts audio de révision
- 121. Flashcards avec répétition espacée
- 122. Cours complémentaires avec des professeurs partenaires
- 123. Bibliothèque de sujets corrigés rédigés par d'anciens admis
- 124. Recommandation de ressources externes fiables

Total : 124 fonctionnalités. Elles ne sont évidemment pas à construire en même temps – voir la feuille de route phasée plus bas.

5. IA fonctionnelle — comment la rendre réelle et pas un gadget

Pour que ce ne soit pas juste un mot à la mode :

- **Génération de QCM** : upload d'un PDF de cours → extraction du texte → appel à un LLM (API Claude ou équivalent) pour générer des questions avec 4 propositions, une bonne réponse, une explication. Un contributeur relit et valide avant publication — l'IA propose, l'humain dispose.
- **Correction de rédaction** : l'utilisateur soumet un texte, l'IA évalue structure, argumentation, orthographe, et donne un retour constructif (pas juste une note).
- **Tuteur conversationnel** : basé sur le contenu réel de tes fiches de cours (pas de réponses inventées), pour éviter les hallucinations sur des notions scolaires précises.
- **Coût** : dans un contexte où les utilisateurs paient peu, prévois un usage raisonné de l'IA (mise en cache des générations, limite d'appels en offre gratuite, IA réservée aux abonnés premium ou aux contributeurs).

6. Nouvelle direction design

- Palette moderne et énergique plutôt que la déclinaison classique bleu/blanc "site administratif" — quelque chose qui évoque la réussite et la progression (dégradés, accents vifs sur fond neutre).
- Dashboard centré sur la progression personnelle dès la connexion (pas un simple menu de liens).
- Dark mode natif, pas un simple filtre CSS.
- Typographie très lisible en petite taille (beaucoup d'usage sur petit écran mobile).
- Animations discrètes qui donnent du feedback (réponse correcte/incorrecte, montée de niveau) sans ralentir l'appareil.
- Mode "focus examen" : interface totalement épurée, sans menu ni distraction, pendant le passage d'un examen blanc.

Si tu veux, je peux te construire un prototype interactif (HTML/React) de cette nouvelle interface pour que tu visualises concrètement — dis-moi et je le fais dans un prochain message.

7. Feuille de route par phases

Phase 1 — Fondations (2 à 4 semaines)

Corriger le rendu SSR/SEO, sécuriser le calcul du score côté serveur, structurer par niveau

(BEPC/Bac/Licence), mettre en place tests + monitoring de base.

Phase 2 – Engagement (2 à 3 mois)

Gamification (XP, badges, streaks), IA basique (génération de QCM, explications), PWA/offline, paiement Mobile Money.

Phase 3 – Communauté & IA avancée (3 à 6 mois)

Fonctionnalités sociales, tuteur IA conversationnel, correction de rédaction, marketplace de contenu.

Phase 4 – Passage à l'échelle (6 à 12 mois et +)

Application mobile native, API publique, multilingue, partenariats établissements scolaires, version SMS/USSD.

8. Garantir la qualité dans la durée

- Tests automatisés systématiques avant chaque déploiement (non négociable dès que le nombre d'utilisateurs grandit).
- Boucle de feedback utilisateur intégrée (bouton "signaler un problème" visible partout).
- Bêta-testeurs réels (des candidats aux concours) avant chaque grosse mise à jour.
- Revue de code régulière, même si tu es seul sur le projet — se relire à froid quelques jours après évite beaucoup d'erreurs.

9. Prochaines étapes concrètes

J'ai maintenant lu ton code réel (voir section 1). Dans l'ordre de priorité que je recommande :

1. **Aujourd'hui** : passe le dépôt en privé le temps de nettoyer `upload/` , `skills/` et `db/custom.db` (risque légal + poids inutile).
2. **Cette semaine** : retire `ignoreBuildErrors` / `ignoreDuringBuilds` de `next.config.ts` , ajoute un script `test` , mets en place une CI GitHub Actions minimale.
3. **Ce mois-ci** : corrige le contrôle d'autorisation manquant sur la route des réponses, aligne le champ `level` sur BEPC/Bac/Licence et commence à combler le contenu collège/lycée.
4. Dis-moi lequel de ces chantiers tu veux qu'on attaque en premier en détail (je peux te fournir le code des correctifs directement), ou si tu préfères qu'on parte sur la maquette de la nouvelle interface.